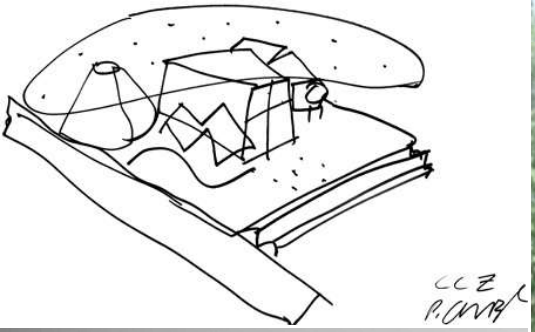
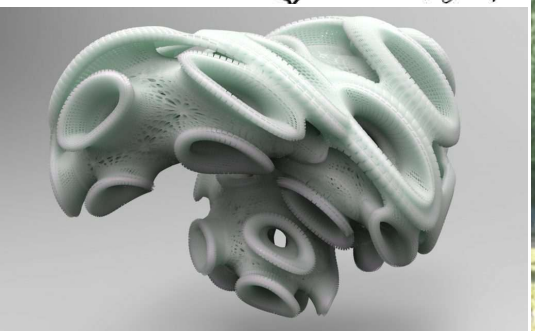






top left: Hand sketch of the Zarautz Scenic Arts Center by Coop Himmelb(l)au / bottom left: Computational form generated with the Grasshopper plugin Pufferfish by Michael Pryor / right: ClipHut, TH OWL

MID 3010 / Elective - Advanced Programming / WiSe 26/27

LOOPHOLE

> NOTICE
> Next class on Wednesday, 14 October, from 9:00 am @ Raum 4.107 All Classes henceforth will be conducted in Raum 4.106

CONTEXT
Grasshopper has no loops. Data flows one way through a definition, from left to right, and the graph may never feed back into itself. Yet almost everything a designer wants to grow, pack, relax, subdivide or optimise is iterative: the result of one step is the input of the next.

For more than a decade the community has closed this gap with third-party add-ons. The best known of them, Anemone, sits inside countless definitions — it is closed-source and has not seen a release in years. Whoever depends on it cannot read it, repair it or learn from it.

This course takes the gap as its brief. Under the working title Loophole, the seminar designs, builds and publishes an open-source loop plugin for Grasshopper: a deliberate loophole in the rule that a definition must not feed back into itself.

CONTENT
The course runs as one continuous software project with a public deadline: at the end of January the plugin is released on Food4Rhino, the platform for Rhino and Grasshopper add-ons, where anyone can install it.

Five subjects are covered along the way:

- Plugin anatomy — how a Grasshopper component is defined, built, packaged and published, read directly from the developer documentation
- Version control — Git and GitHub as the working environment of a team: repositories, branches, pull requests, code review, issues
- The solver — how Grasshopper computes a definition, why it refuses cycles, and the three ideas a loop is built from: expiry, scheduling and state
- Software architecture — how one tool is divided among many authors so that the parts fit and nobody blocks anybody
- Release engineering — packaging, versioning, documentation, licence, and the listing that puts the plugin in front of strangers

Prototypes are written in Grasshopper's script components, in Python or C#. The shared plugin ships as a Grasshopper assembly (.gha) through the Rhino package manager and Food4Rhino; its language is decided in Session 9 on the basis of the individual solutions.

METHODOLOGY

The course begins with the sources rather than with tutorials. Students study the Grasshopper developer documentation to understand how a plugin is built and published on Food4Rhino, and learn Git and GitHub as the working environment for everything that follows. The solver is taught as vocabulary, not as a recipe: what expiry, scheduling and state mean, not how to assemble them into a loop. Each student then develops their own solution for loop functions in Grasshopper. No approach is prescribed; different answers are wanted, and each has to be argued for. In every second session each student walks the group through their own code — the architecture, the definitions and functions used, and what broke since last time. The individual solutions are presented and discussed in a joint session, from which the seminar decides on a common architecture and direction. From then on the whole class, supervisors included, builds the plugin together in one shared repository, in seminar sessions with regular review; the walkthroughs continue, now on the shared code. The code is written by hand, with minimal use of AI, so that every contributor can explain what they commit. The process is documented in the repository throughout, and the result is presented and published at the end of January.

GOALS

The course aims at authorship, not at the operation of tools.

Three capabilities are to be established. First, the ability to read technical documentation and build from it without a tutorial. Second, the ability to work in a shared codebase: to make changes that others can review, and to review the changes of others. Third, the experience of carrying software all the way to release — packaged, documented and installed by people you will never meet.

Behind all three stands a change of position: from user of Grasshopper plugins to author of one.

REQUIREMENTS

Rhino 8 with Grasshopper, a GitHub account, and a laptop on which you may install software: Git, and the .NET SDK or Visual Studio 2022 Community. Everything is installed together in Session 1. Two semesters of Python and Grasshopper are assumed; C# is not, and is learned where it is needed.

SUBMISSION

T1 LOOP FUNCTIONS [Individual Submission] — 2 Dec 26, 9:00 am, Presentation in class

T2 PLUGIN RELEASE [Group Submission] — 27 Jan 27, 9:00 am, Final presentation + Food4Rhino release

SUPERVISION

Prof. Hans Sachs, Hrithik Shetty

LOCATION

R 4.106, Detmold Campus · 09:00–12:00



Zaha Hadid Architects, Photo: Xia Shi

MID 3010 / Elective - Advanced Programming / WiSe 26/27

SCHEDULE

7 Oct 26

Session 1

LECTURE Introduction: why Grasshopper has no loops
SEMINAR Developer documentation: anatomy of a component, from .gha to Food4Rhino
SETUP Rhino 8, Git, GitHub account, .NET SDK, first script component

14 Oct 26

Session 2

LECTURE Git and GitHub I: repositories, commits, branches
SEMINAR One repository per student, first commits, README
In Raum 4.107

21 Oct 26

Session 3

LECTURE Git and GitHub II: pull requests, code review, issues
LECTURE The solver: expiry, scheduling, state
SEMINAR Release TASK 1, first experiments

28 Oct 26

Session 4

SEMINAR Individual loop solutions: approaches, first prototypes
INDIVIDUAL REVIEWS

4 Nov 26

T1 FINAL PRESENTATION [INDIVIDUAL]

PRESENTATION Individual loop solutions (TASK 1)
DISCUSSION Architecture, language and component list of the common plugin nodes to include. Start planning the code architecture for TASK 2

11 Nov 26

Session 5

SEMINAR Plugin build: repository, scaffold, component ownership, definition of done
GROUP REVIEW

18 Nov 26

~~Session 6~~

SEMINAR Plugin build: repository, scaffold, component ownership, definition of done

— Cancelled

GROUP REVIEW
Session Cancelled due to RESET/Schuco Conference

25 Nov 26	**Session 6** SEMINAR Plugin build: repository, scaffold, component ownership, definition of done GROUP REVIEW
2 Dec 26	**Session 7** SEMINAR Plugin build: repository, scaffold, component ownership, definition of done GROUP REVIEW
9 Dec 26	**Session 8** SEMINAR Plugin build: repository, scaffold, component ownership, definition of done GROUP REVIEW
16 Dec 26	**Session 9** SEMINAR Plugin build: repository, scaffold, component ownership, definition of done GROUP REVIEW
23 Dec 26	No class Christmas Holidays
30 Dec 26	No class Christmas Holidays
6 Jan 27	**Session 10** SEMINAR Plugin build: components, pull requests, cross-review GROUP REVIEW
13 Jan 27	**Session 11** CODE WALKTHROUGH IV: integration — how the components fit, what the example files test SEMINAR Integration and testing on real definitions
20 Jan 27	**Session 12** SEMINAR Release engineering: package, icons, licence, documentation, example files, listing GROUP REVIEW Release candidate, feature freeze
27 Jan 27	**Final Submission** PRESENTATION and RELEASE Publication of the plugin on Food4Rhino

---

Last updated: 13 Oct 2026, 19:21